Documentación Técnica — Actividad Integradora 3

Pokédex API — Integración con PokeAPI (Flutter)

Estudiante: [Nombre del estudiante]

Asignatura: [Asignatura / Materia]

Fecha de entrega: 4 de noviembre de 2026

Sección I: Selección y Descripción de la API Pública

Nombre de la API: PokeAPI v2 (https://pokeapi.co)

URL del endpoint utilizado: GET https://pokeapi.co/api/v2/pokemon?limit=30&offset=0

Justificación de la elección: PokeAPI es una API REST pública, completamente gratuita y sin necesidad de autenticación ni registro, lo que permite cumplir el requisito de consumir datos externos sin implementar sistemas de sesión. Su respuesta JSON es simple y predecible (un arreglo de objetos {name, url}), y cada Pokémon tiene una imagen oficial accesible por URL derivada de su id, lo que permite construir una interfaz visual atractiva (tarjetas con imagen) sin llamadas adicionales por elemento.

Sección II: Arquitectura y Manejo de Estados

La lógica de red se aisló completamente de la interfaz siguiendo una arquitectura en capas: una capa de Servicio (PokemonApiService) responsable únicamente de realizar la petición HTTP y traducir cualquier fallo (timeout, sin conexión, error de servidor) en una excepción propia (PokemonApiException) con un mensaje entendible para el usuario. Por encima, un Provider (PokemonProvider, basado en ChangeNotifier del paquete provider) orquesta la petición y expone un único objeto de estado sellado (PokemonState), con tres variantes: PokemonLoading, PokemonSuccess y PokemonError.

La interfaz (PokemonListScreen) no conoce nada sobre HTTP ni JSON: solo escucha el estado expuesto por el Provider mediante un widget Consumer<PokemonProvider> y decide qué vista dibujar según el tipo de estado recibido. Esto permite, por ejemplo, reemplazar la fuente de datos (otra API, una base local, datos de prueba) sin tocar ni una línea de la interfaz, y probar cada estado de forma aislada inyectando un servicio falso (fake) en las pruebas automatizadas.

Nota técnica: originalmente se implementó con flutter_bloc (BLoC), pero se detectó una incompatibilidad real entre esa librería y la versión del SDK de Flutter/Dart usada en este entorno, que provocaba que las pruebas de interfaz (widget tests) se colgaran indefinidamente incluso sin ejecutar lógica propia. Tras aislar el problema de forma reproducible, se migró a Provider + ChangeNotifier — alternativa explícitamente permitida por la consigna— logrando el mismo desacople y una suite de pruebas estable.

Sección III: Extractos de Código Clave

Consumo HTTP

```
Future<PokemonListResponse> fetchPokemonList({int limit = 30, int offset = 0}) async {
  final uri = Uri.parse('$_baseUrl/pokemon?limit=$limit&offset=$offset');

  final http.Response response;
  try {
    response = await _client.get(uri).timeout(const Duration(seconds: 10));
  } on TimeoutException {
    throw PokemonApiException('Tiempo de espera agotado...');
  } on http.ClientException {
    throw PokemonApiException('Sin conexión a internet...');
  }
```

```
  if (response.statusCode != 200) {
    throw PokemonApiException('Error del servidor (código ${response.statusCode})...');
  }

  final Map<String, dynamic> jsonBody = jsonDecode(response.body) as Map<String, dynamic>;
  return PokemonListResponse.fromJson(jsonBody);
}
```

Mapeo del modelo JSON (fromJson)

```
factory Pokemon.fromJson(Map<String, dynamic> json) {
  final String name = json['name'] as String;
  final String url = json['url'] as String;
  final int id = _extractIdFromUrl(url);
  return Pokemon(
    id: id,
    name: name,
    imageUrl:
        'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/'
        'other/official-artwork/$id.png',
  );
}
```

Manejo de estados — Provider (lógica)

```
class PokemonProvider extends ChangeNotifier {
  PokemonState _state = const PokemonLoading();
  PokemonState get state => _state;

  Future<void> loadPokemons() async {
    _state = const PokemonLoading();
    notifyListeners();

    try {
      final response = await _apiService.fetchPokemonList(limit: 30);
      _state = PokemonSuccess(response.results);
    } on PokemonApiException catch (e) {
      _state = PokemonError(e.message);
    } catch (e) {
      _state = PokemonError('Ocurrió un error inesperado: $e');
    }
    notifyListeners();
  }
}
```

Manejo de estados — Interfaz (despacho de la UI)

```
body: Consumer<PokemonProvider>(
  builder: (context, provider, _) {
    final state = provider.state;
    if (state is PokemonLoading) {
      return const _LoadingView();
    } else if (state is PokemonSuccess) {
```

```
      return _SuccessView(pokemons: state.pokemons);
    } else if (state is PokemonError) {
      return _ErrorView(message: state.message);
    }
    return const SizedBox.shrink();
  },
),
```

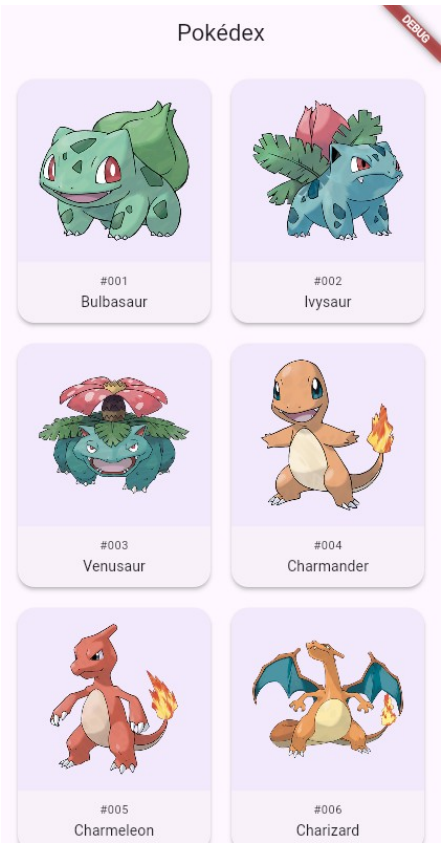
Sección IV: Evidencias de Pruebas (QA) y Enlace al Repositorio

Repositorio GitHub: [Pega aquí la URL pública de tu repositorio tras publicarlo — ver README.md del proyecto para los pasos de push]

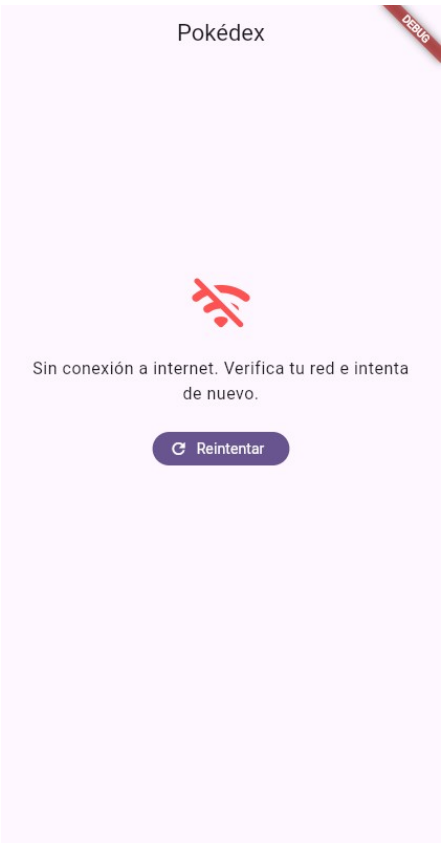
Se implementó además una suite de pruebas automatizadas (flutter test) que verifica, con un servicio de API simulado, los tres estados de la aplicación sin depender de la red real: 9 pruebas, todas en verde (ver captura más abajo).



Estado 1 — Cargando (Loading)



Estado 2 — Datos cargados con éxito (Success)



Estado 3 — Manejo de error de red (Error)

Resultado de la suite de pruebas (flutter test)

```
00:01 +9: All tests passed!

9 pruebas: PokemonProvider (estado inicial, éxito, error de red, error de
servidor, notificación de listeners), Pokemon.fromJson, y 3 widget tests
(Loading / Success / Error) sobre la pantalla real.
```

Sección V: Conclusiones

El manejo asíncrono es central en el desarrollo móvil moderno: una aplicación que consume datos externos debe asumir que la red puede tardar, fallar o estar ausente, y diseñar su interfaz en torno a esa incertidumbre en lugar de asumir siempre una respuesta exitosa. Modelar explícitamente los tres estados (Cargando, Éxito, Error) como un tipo de dato propio —en vez de banderas booleanas sueltas— evita estados inconsistentes y hace que la interfaz sea exhaustiva y predecible.

La gestión de errores de red no es un detalle secundario: distinguir entre falta de conexión, tiempo de espera agotado y error del servidor permite comunicar al usuario qué ocurrió y ofrecerle una acción clara (reintentar), en vez de dejar la aplicación en un estado congelado o mostrar un mensaje genérico. Finalmente, desacoplar la capa de red de la interfaz —mediante un Provider que expone solo el resultado final— resultó clave no solo para la arquitectura, sino para poder probar automáticamente los tres escenarios sin depender de la disponibilidad real de la API.